

C++ 프로그래밍 및 실습 Tic Tac Toe 실습 보고서

인공지능학부 223274 김영광

1. 서론

1-1. 프로젝트 목적 및 배경: 4주차까지 배운 배열 및 반복문을 이용한 틱택토 게임 실습

1-2. 목표: Tic Tac Toe 게임 기능 구현 및 실습

2. 요구사항

2-1. 사용자 요구사항: 두 명의 사용자가 $n \times n$ 배열 보드에 O와 X를 놓아 빙고를 만든 사용자가 승리하는 게임

2-2. 기능 요구사항:

- 누구 차례인지 출력
- 좌표 입력 받기
- 입력 받은 좌표 유효성 체크
- 좌표에 O/X 놓기
- 현재 보드판 출력
- 빙고 시 승자 출력 후 종료
- 모든 칸이 찼으면 종료

3. 설계 및 구현

3-1. 누구 차례인지 출력

```
while (true){
    //1. 누구 차례인지 출력
    switch (k % 2){
        case 0:
            cout << k % 2 + 1 << "번 유저(x)의 차례입니다 -> ";
            currentUser = 'x';
            break;
        case 1:
            cout << k % 2 + 1 << "번 유저(o)의 차례입니다 -> ";
            currentUser = 'o';
            break;
    }
}
```

입력

- k = 각 플레이어 차례 값

결과

- 각 플레이어 차례마다 1을 더하여 플레이어 구분

설명

- k 값이 짝수면 유저(x), 홀수이면 유저(o)

3-2. 좌표 입력 받기

```
//2. 좌표 입력 받기
cout << "(x, y) 좌표를 입력하세요: ";
cin >> x >> y;
```

입력

- x = 좌표 x 값, y = 좌표 y 값

설명

입력 받을 각 좌표 값

3-3. 입력 받은 좌표 유효성 체크

```
//3. 입력받은 좌표의 유효성 체크
if (x >= numCell || y >= numCell){
    cout << x << ", " << y << ": ";
    cout << " x와 y 둘 중 하나가 칸을 벗어납니다. " << endl;
    continue;
}
if (borad[x][y] != ' '){
    cout << x << ", " << y << ": 이미 돌이 차있습니다." << endl;
    continue;
}
```

입력

x = x 좌표 값,

y = y 좌표 값

numCell = 가로/세로 칸 개수

결과

- 칸을 놓을 수 이유 출력

설명

- 사용자가 칸을 벗어나서 입력인지
체크

- 이미 돌이 있는 칸에 입력했는지
체크

3-4. 좌표에 O/X 놓기

```
//4. 입력받은 좌표에 현재 유저의 돌 놓기
borad[x][y] = currentUser;
```

입력

- borad[x][y] = n x n 2차열 배열

결과

- 현재 입력 받은 값을 각 배열
입력

3-5. 현재 보드판 출력

```
//5. 현재 보드 판 출력
for (int i = 0; i < numCell; i++) {
    cout << "---|---|---" << endl;
    for (int j = 0; j < numCell; j++){
        cout << borad[i][j];
        if (j == numCell - 1) {
            break;
        }
        cout << "  |";
    }
    cout << endl;
}
cout << "---|---|---" << endl;
```

결과

- 각 보드에 있는 값을
2중 for문으로 출력

설명

- 한줄 씩 입력한 값 작성 후 구분선 출력

3-6. 빙고 시 승자 출력 후 종료

a. 가로 조건

```
//가로 검증
if (borad[0][0] == borad[0][1] && borad[0][1] == borad[0][2] && borad[0][1] == currentUser){
    cout << "가로에 모두 들어 놓았습니다!: " << k % 2 + 1 << "번 유저의(" << currentUser << ")의 승리입니다!";
    break;
}
if (borad[1][0] == borad[1][1] && borad[1][1] == borad[1][2] && borad[1][1] == currentUser){
    cout << "가로에 모두 들어 놓았습니다!: " << k % 2 + 1 << "번 유저의(" << currentUser << ")의 승리입니다!";
    break;
}
if (borad[2][0] == borad[2][1] && borad[2][1] == borad[2][2] && borad[2][1] == currentUser){
    cout << "가로에 모두 들어 놓았습니다!: " << k % 2 + 1 << "번 유저의(" << currentUser << ")의 승리입니다!";
    break;
}
```

b. 세로 조건

```
//세로 검증
if (borad[0][0] == borad[1][0] && borad[1][0] == borad[2][0] && borad[1][0] == currentUser){
    cout << "세로에 모두 돌이 놓였습니다!: " << k % 2 + 1 << "번 유저의(" << currentUser << ")의 승리입니다!";
    break;
}
if (borad[0][1] == borad[1][1] && borad[1][1] == borad[2][1] && borad[1][1] == currentUser){
    cout << "세로에 모두 돌이 놓였습니다!: " << k % 2 + 1 << "번 유저의(" << currentUser << ")의 승리입니다!";
    break;
}
if (borad[0][2] == borad[1][2] && borad[1][2] == borad[2][2] && borad[1][2] == currentUser){
    cout << "세로에 모두 돌이 놓였습니다!: " << k % 2 + 1 << "번 유저의(" << currentUser << ")의 승리입니다!";
    break;
}
```

c. 대각선 조건

```
if (borad[0][0] == borad[1][1] && borad[1][1] == borad[2][2] && borad[1][1] == currentUser){
    cout << "대각선에 모두 돌이 놓였습니다!: " << k % 2 + 1 << "번 유저의(" << currentUser << ")의 승리입니다!";
    break;
}
if (borad[0][2] == borad[1][1] && borad[1][1] == borad[2][0] && borad[1][1] == currentUser){
    cout << "대각선에 모두 돌이 놓였습니다!: " << k % 2 + 1 << "번 유저의(" << currentUser << ")의 승리입니다!";
    break;
}
```

3-7. 모든 칸이 찼으면 종료

```
//모든 칸이 다 찬 경우 종료
if (count >= numCell * numCell){
    cout << "모든 칸이 다 찼습니다. 종료합니다.";
    break;
}
```

입력

- count 사용자가
돌을 놓은 횟수

결과

- 사용자가 빙고
반의 개수보다
많이 놓으면 종
료

4. 테스트

4-1. 누구 차례인지 출력

```
gloryrla@jim-yeong-gwang-u1-MacBookAir week5 % cd "/Users/gloryrla/CPP-26-2/project/week5/"tic_tac_toe
1번 유저 (x)의 차례입니다 -> (x, y) 좌표를 입력하세요 : $
```

4-2. 좌표 입력 받기

```
2번 유저 (o)의 차례입니다 -> (x, y) 좌표를 입력하세요 :
```

4-3. 입력 받은 좌표 유효성 체크

```
s7gloryrla/CPP-26-2/project/week5/ tic_tac_toe
1번 유저 (x)의 차례입니다 -> (x, y) 좌표를 입력하세요 : 0 0
---|---|---
x  |   |   |
---|---|---
   |   |   |
---|---|---
   |   |   |
2번 유저 (o)의 차례입니다 -> (x, y) 좌표를 입력하세요 : 0 0
0, 0: 이미 둘이 차있습니다.
```

4-4. 좌표에 O/X 놓기

```
2번 유저 (o)의 차례입니다 -> (x, y) 좌표를 입력하세요 : 1 1
---|---|---
x  |   |   |
---|---|---
   |o  |   |
---|---|---
   |   |   |
```

4-5. 현재 보드판 출력

```
---|---|---
x  |   |   |
---|---|---
   |o  |   |
---|---|---
   |   |x  |
---|---|---
```

4-6. 빙고 시 승자 출력 후 종료

```
2번 유저 (o)의 차례입니다 -> (x, y) 좌표를 입력하세요 : 2 1
---|---|---
x  | o  |
---|---|---
x  | o  |
---|---|---
   | o  | x
---|---|---
세로에 모두 둘이 놓였습니다!: 1번 유저의 (x)의 승리입니다!%
```

4-7. 모든 칸이 찼으면 종료

```
1번 유저 (x)의 차례입니다 -> (x, y) 좌표를 입력하세요 : 2 1
---|---|---
x  | o  | o
---|---|---
o  | x  | x
---|---|---
x  | x  | o
---|---|---
모든 칸이 다 찼습니다. 종료합니다. %
```

5. 결과 및 결론

5-1. 프로젝트 결과: Tic Tac Toe 게임 완성, 세부 기능 요구사항 구현 및 테스트 검증

5-2. 느낀 점: 처음부터 반복문으로 하려고 하니깐 머리가 너무 아팠습니다. 우선은 반복문 작성 전에 하나씩 작성하고 반복하는 코드들을 반복문으로 작성하는 게 더 직관적이어서 먼저 코드들을 작성하고 나중에 반복되는 코드들을 반복문으로 작성해야겠다고 느꼈습니다. 3명이서 하는 Tic Tac Toe 게임은 반복문을 사용하는 코드로 작성해서 제출해 보고 싶어졌습니다.